Design and Evaluation of a Raspberry Pi–Based Vision System for Yarn Error Detection on a Custom Knitting Machine

Alessia Rinaldi

This manuscript was compiled on May 4, 2026

Ensuring reliable quality control in crochet and knitting based textile production is particularly challenging for industrial machines, where defects such as missing yarn or holes can occur at the scale of individual hooks and propagate over many cycles before they are noticed and lead to significant waste of material. This paper presents a complete low-cost embedded vision system for real-time monitoring of an industrial crochet machine, with the specific objective of verifying, at each hook movement, that the yarn correctly wraps around the hook and as soon as an anomaly is detected automatically stopping the machine. The proposed solution is built around a Raspberry Pi 3 Model B+ equipped with a Pi Camera module connected via the CSI interface and a microswitch provides a hardware synchronization signal. All mounted on a custom 3D-printed support designed to satisfy the mechanical constraints of the crochet machine and to ensure a stable view of the region of interest. Images are acquired in synchrony with the machine motion, pre-processed to isolate the relevant area, and analysed by a dedicated detection algorithm to identify missing yarn or holes. When an error is detected, the system triggers an immediate stop command to the machine. This paper describes the hardware selection, the mechanical integration, and the software pipeline and evaluates the system on data collected in an industrial scenario, reporting its performance in terms of false positives and false negatives. The results demonstrate the feasibility of using low-cost embedded vision for inline quality control on crochet machines and highlight directions for further improvements.

Textile Defect Detection | Embedded Vision | Quality Control | Real-Time Monitoring

1. Introduction

Textile practices such as crochet and knitting have recently experienced a widespread resurgence. Many creative activities traditionally rooted in manual craftsmanship, are now being rediscovered. Skills that historically required years of hands-on experience can, however, be difficult and discouraging for beginners, especially for adults who approach these practices with limited time, resources, or access to expert guidance. Crochet is a notable example: although widely practiced across cultures, it remains technically demanding and requires the maker to continuously recognize and correct errors. This challenge is further amplified when using hobbyist tools such as circular crochet looms, which are widely available and inexpensive but provide no feedback on errors and therefore require constant user attention.

In this work, we start from a commercially available circular loom intended for hobby use and, through a series of mechanical, electronic, and software modifications, transform it into a fully automated crochet machine capable of self-monitoring. This approach demonstrates how accessible off-the-shelf tools can be augmented and repurposed into automated systems, lowering the barrier to entry for creative technologies.

A single undetected defect, such as missing yarn, incomplete engagement, or a misaligned loop, may propagate across many subsequent cycles, ultimately compromising the entire piece. For individual makers and hobbyists, failing to detect such an error in time can mean that hours of work are lost and the product must be unraveled or discarded. This results not only in wasted material but also in significant frustration for the user, particularly in creative practices where the making process is closely tied to personal time and effort.

Automating the early detection of such defects requires monitoring the hook at the precise moment when the yarn configuration becomes informative. In this context, the error detection mechanism must remain non-intrusive. Any sensing element that physically interacts with the yarn could interfere with the stitch formation process and introduce additional disturbances. For this reason, a vision-based approach was adopted, enabling the system to monitor the yarn configuration without altering the mechanical behavior of the crochet process. In the scenario addressed in this work, a correctly formed stitch appears as a characteristic "U"-shaped engagement around the base of the hook. Any deviation from this expected pattern must be identified promptly to prevent the defect from propagating downstream.

In this paper, we present a lightweight, real-time embedded vision system for hook-level error detection implemented entirely on a Raspberry Pi 3 equipped with a CSI camera module and mounted on a custom 3D-printed support. The system captures images in synchrony with the machine's motion, isolates the region of interest, and analyzes it using a dedicated detection algorithm capable of distinguishing correct yarn engagement from error conditions. Our goal is to extend accessible hobbyist tools with automated monitoring capabilities, demonstrating how low-cost hardware and embedded vision can support makers during the crochet process.

2. Selection of the Acquisition System

The acquisition system plays a central role in the proposed error detection pipeline, as it must provide close-up images of the hook region at every relevant movement of the crochet machine. Since the yarn manipulation process is highly sensitive to external disturbances, the sensing solution must remain non-intrusive, which motivates the use of a vision-based acquisition system. In this work, we start from a commercially available circular crochet loom intended for hobby use and adapt it into an automated system through a set of mechanical and electronic modifications. This choice allows us to preserve the low-cost and accessible nature of the original tool while enabling precise synchronization and repeatable inspection of

each hook. An overview of the original circular crochet loom is shown in Fig. 1.

Fig. 1. Overview of the circular crochet loom.

To synchronize image acquisition with the machine motion, a hardware microswitch was mounted on the loom, as shown in Fig. 2. The switch generates a synchronization signal that marks the exact moment in the hook's motion when the yarn configuration becomes informative. This signal is used by the Raspberry Pi 3 Model B+ to trigger image acquisition in real time.

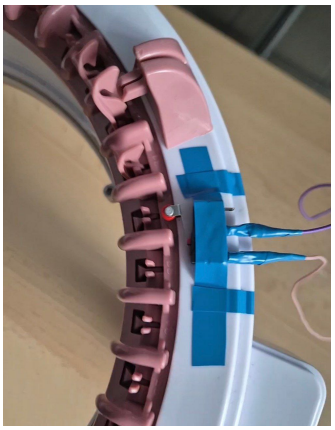


Fig. 2. Placement of the microswitch on the circular crochet loom used to detect the formation of a new stitch and trigger image acquisition.

At each trigger, the system acquires an image of the hook region and inspects the area where the stitch is expected to form. Figure 3 shows an example frame captured by the Pi Camera, together with the region of interest used for the analysis. Within this region, the system verifies whether the yarn forms the expected "U"-shaped configuration around the base of the hook, independently of the yarn color.

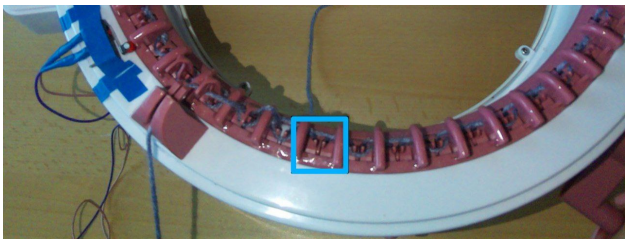


Fig. 3. Example image captured by the Pi Camera. The light-blue rectangle highlights the region of interest where the characteristic U-shaped yarn engagement is expected to appear.

Examples of correct and defective yarn configurations are reported in Fig. 8. Any configuration that does not match the expected pattern is classified as an error. When an anomaly is detected, the system immediately stops the motor driving the process and displays a diagnostic message indicating the index of the affected hook.

(a) Positive example **(b)** Negative example

Fig. 4. Examples of correct (left) and defective (right) yarn configuration.

A. Requirements. The primary objective of the acquisition system is to enable automatic, hook-level monitoring of the knitting process on a low-cost embedded platform. To meet this goal, the choice of camera and interface must satisfy the following requirements:

- **Compatibility with Raspberry Pi 3:** use of natively supported interfaces and libraries.
- **Ease of software integration:** availability of well-established computer vision libraries (e.g., libcamera, Picamera2, OpenCV).
- **Cost-effectiveness:** a sensor with sufficient resolution to distinguish fine yarn details while keeping costs low.
- **Close-focus capability:** ability to capture sharp images at short distances near the hook area.
- **Low power consumption:** efficient power usage to minimize the overall system load.

These requirements guided the evaluation of several acquisition architectures for the Raspberry Pi 3, considering their impact on image quality, latency, software complexity, and mechanical integration.

B. Considered Acquisition Architectures.

B.1. Pi Camera via CSI Interface (Selected Option). The Raspberry Pi Camera v2.1 connects directly to the Broadcom SoC through the MIPI–CSI interface, a high-speed differential serial bus dedicated to video capture. This direct link offers several advantages for the considered application:

- High bandwidth and low latency;
- Image quality and control;
- Native compatibility with Raspberry Pi 3.

In the context of hook-level defect detection, these properties translate into a favorable signal-to-noise ratio and consistent frame characteristics, which are beneficial for detecting subtle differences in yarn configuration. The low and predictable latency also facilitates accurate synchronization between the microswitch event and the captured image, which is critical for reliable inspection.

B.2. ESP32-CAM via Wi-Fi or Serial. The ESP32-CAM module integrates an OV2640 sensor (2 MP) and a microcontroller with on-board Wi-Fi connectivity. In a typical setup, images are captured on the ESP32-CAM and transmitted to the Raspberry Pi for processing, either via Wi-Fi or over a serial link:

- **MQTT-based variant (unsuccessful):** A Wi-Fi-based architecture using the ESP32-CAM was also evaluated, where images were periodically captured and transmitted via MQTT to a Raspberry Pi running a Mosquitto broker. However, this approach proved unreliable due to frequent decoding errors and corrupted frames during JPEG transmission. As a result, the ESP32-CAM solution was deemed unsuitable for the timing and robustness requirements of hook-level error detection, and the development effort was redirected toward more tightly integrated alternatives.
- **Serial variant (unsuccessful):**In preliminary experiments with a direct USB–UART connection, the ESP32-CAM was successfully flashed with a MicroPython firmware. However, upon reboot, the board remained stuck in the bootloader, and the serial output consisted only of unreadable characters or repetitive error messages, never reaching a stable REPL prompt. This behavior prevented the validation of a reliable point-to-point communication channel with the Raspberry Pi and hindered further development of the serial-based approach.

The ESP32-CAM is extremely low cost and can operate as a stand-alone node. However, compared to a CSI-connected Pi Camera, it provides lower image quality and less precise control over acquisition parameters. From a mechanical standpoint, the ESP32-CAM module is also bulkier than the Pi Camera, which complicates its integration into the compact 3D-printed frame of the desktop crochet machine. On the positive side, its shorter minimum focus distance can be advantageous for very close-up shots, potentially reducing the need for additional optics.

B.3. Other Considered Alternatives (Not Tested). Two further options were considered at the design stage but not experimentally tested.

- **USB webcam:**Standard USB webcams offer a ready-to-use and widely available solution, typically with UVC support under Linux. However, image data must traverse the USB stack, introducing additional overhead compared to CSI and limiting the level of low-level camera control. Such devices can be suitable for early prototyping or applications with less stringent latency requirements, but they are less attractive when deterministic timing and compact integration are needed.
- **Raspberry Pi HQ Camera:**The Raspberry Pi HQ Camera is a 12 MP module with a C/CS mount for interchangeable lenses. It provides superior optical quality and flexibility in terms of focus, focal length, and dedicated macro optics. These advantages come at the cost of higher price and significantly larger size compared to the Pi Camera v2.1. The HQ Camera is therefore more appropriate when extreme close-ups or professional optical control are required, whereas the present work prioritizes low cost and compactness for a hobby-grade desktop machine.

C. Final Choice. Considering the functional requirements, the practical experiments with the ESP32-CAM, and the constraints of the crochet machine, the Pi Camera v2.1 connected via the MIPI–CSI interface emerged as the most suitable option. It combines sufficient resolution and close-focus capability with low and deterministic latency, strong software support on the Raspberry Pi 3, and a compact form factor compatible with the 3D-printed camera mount described in Section 3. This configuration is therefore adopted as the reference acquisition system for the error detection pipeline.

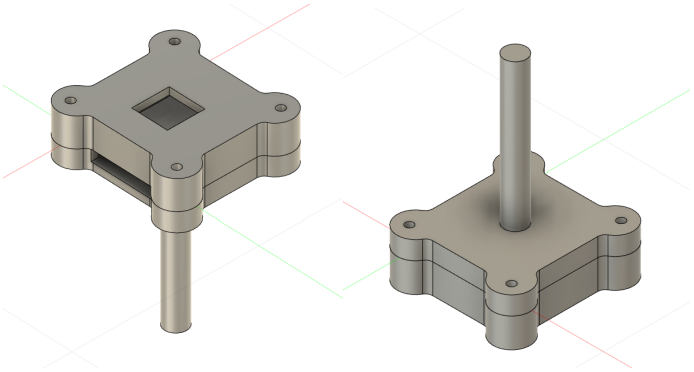
3. Mechanical Design of the Camera Mount

The camera mounting solution combines a commercially available magnetic-base stand with a custom 3D-printed support designed specifically for the Raspberry Pi Camera module. The magnetic base provides a rigid and highly stable foundation, while allowing fine manual adjustment in height, angle, and lateral position through its articulated arm.

A custom holder for the Raspberry Pi Camera v2.1 was designed in Autodesk Fusion and fabricated using fused deposition modeling (FDM) with PLA filament on a Bambu Lab A1 Mini 3D printer. The printed part encloses the camera PCB and incorporates a series of mounting holes and a pin that fits into the threaded end of the magnetic base arm.

While the magnetic stand and the 3D-printed camera holder provide sufficient stability for prototyping, the overall stiffness of the structure is not optimal yet. Small shifts caused by machine vibration or accidental contact can slightly alter the camera pose over time, affecting the exact position of the region of interest in the image. Improving the mechanical stability of the mounting solution is therefore an important direction for future work, with the goal of achieving fully repeatable alignment over long acquisition periods.

A CAD rendering of the printed holder is shown in Fig. 5. The assembled camera mounting system, including the magnetic base and the 3D-printed holder with the Raspberry Pi Camera installed, is shown in Fig. 6.



(a) Front of the 3D structure **(b)** Back of the 3D structure

Fig. 5. CAD rendering of the custom 3D-printed holder designed for the Raspberry Pi Camera module used to mount the camera on the magnetic-base stand.

4. Error Detection Software and Implementation

The image acquisition and error-recognition pipeline is implemented in Python and runs directly on the Raspberry Pi 3. The system operates in a fully event-driven manner: each

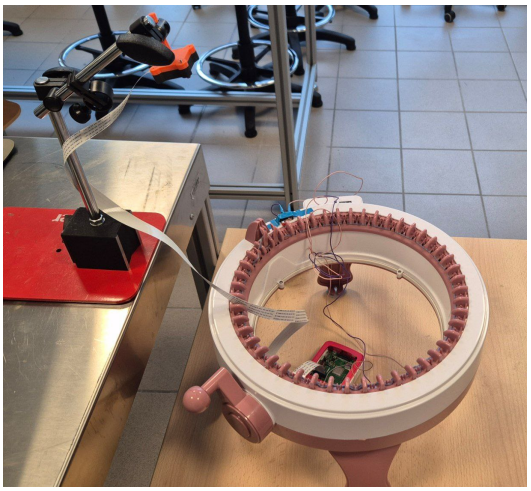


Fig. 6. Photograph of the assembled camera mounting system. The Raspberry Pi Camera module is housed in the custom 3D-printed holder and mounted on the articulated magnetic-base stand used to position the camera above the crochet loom.

activation of the microswitch triggers an image capture, an immediate crop of the region of interest (ROI) and a real-time evaluation of the yarn configuration using a log-likelihood ratio (LLR) classifier.

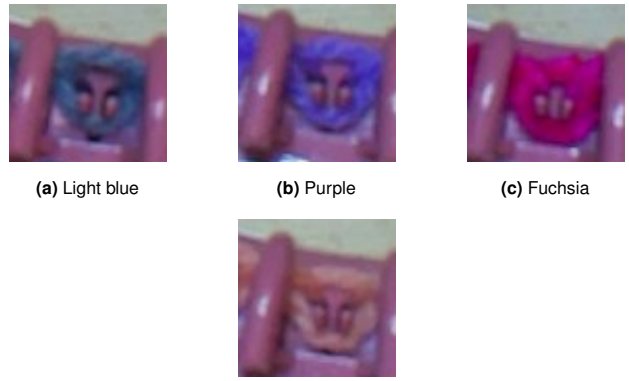
A. Dataset Development and Color-Specific Challenges. To train and evaluate the classifier, a dedicated dataset of cropped hook images was collected directly from the machine. At present, the dataset includes three yarn colors:

- light blue,
- purple,
- fuchsia.

Although these colors appear visually quite different in RGB, their behavior after preprocessing (grayscale conversion, CLAHE enhancement, ROI masking) is not uniform. In fact, preliminary qualitative inspection suggested that the **fuchsia** yarn could be challenging due to its visual similarity to the pink background of the machine. However, subsequent experiments revealed that raw chromatic contrast is not the primary factor affecting detectability in the proposed NCC–LLR pipeline.

In practice, after grayscale conversion the **light blue** yarn becomes closer in intensity to the machine background, resulting in lower separability between correct and defective configurations. Conversely, the **fuchsia** yarn exhibits stronger and more consistent local texture variations within the discriminative ROI, which makes the LLR scores for H_0 and H_1 more distinct despite its low RGB contrast with the background. The **purple** yarn displays intermediate behavior, with moderate grayscale separability.

To further probe the limits of the method, additional experiments were conducted using a **pale pink yarn** whose color is almost indistinguishable from the background. Under this condition, neither the NCC-based similarity nor the CLAHE-enhanced preprocessing provided sufficient discriminative information for reliable classification, and the LLR method consistently failed to separate the two hypotheses. This extreme low-contrast scenario is therefore considered **out of scope for the current handcrafted method** and highlights the need for future solutions based on learned or color-invariant features.



(a) Light blue **(b)** Purple **(c)** Fuchsia **(d)** Pale pink (failure)

Fig. 7. Examples of yarn colors used in the dataset. The pale pink yarn is almost indistinguishable from the machine background and cannot be reliably detected by the current method.

B. Synchronized Acquisition and ROI Extraction. Upon a rising edge from the microswitch (handled via the `gpiozero` library), the software captures a still image using the Picamera 2.1 interface. Since only a small portion of the full frame contains the hook region, the image is immediately cropped according to a predefined ROI. This cropping stage reduces computational cost and increases robustness by ensuring that the classifier operates on a stable and consistent portion of the scene. Boundary checks are applied to avoid invalid crops in case of slight camera misalignment, which is important due to the sub-centimeter positioning precision required for hook-level inspection.

C. Preprocessing and Prototype Generation. The LLR classifier relies on two prototype images:

- a **positive prototype** representing correct yarn engagement;
- a **negative prototype** representing a hook that has failed to catch the yarn.

These prototypes are computed as the median of small training sets of positive and negative examples. Before prototype computation, images undergo grayscale conversion, Gaussian denoising, and light contrast equalization using CLAHE, which improves robustness to illumination variability and sensor noise.

D. Discriminative ROI Mask Estimation. The LLR classifier does not operate on the entire cropped image, but only within a discriminative region of interest (ROI). This ROI is defined by a soft mask computed offline using a separate Python script (`build_mean_diff_mask.py`), which analyzes small pre-cropped patches of both positive and negative examples.

The script takes as input two sets of images: a positive set (`-pos`) containing correctly engaged hooks and a negative set (`-neg`) containing missing-yarn or defective configurations. All images are converted to grayscale, denoised, equalized with CLAHE, and resized to a common resolution of 76×76 pixels. For each pixel location, the script computes the mean and

standard deviation over the positive and negative stacks and derives a Fisher-like separability score:

$$S(i,j) \propto \frac{\left|\mu_{\text{pos}}(i,j) - \mu_{\text{neg}}(i,j)\right|}{\sigma_{\text{pos}}(i,j) + \sigma_{\text{neg}}(i,j) + \varepsilon},$$

The resulting score map is normalized to $[0,1]$ and restricted to a fixed rectangular zone located in the lower part of the patch, corresponding to the physical region where the yarn intersects the hook. Outside this area, the mask is set to zero so that the LLR classifier will completely ignore those pixels.

To preserve local texture and avoid overly fragmented masks, the soft score map S is optionally smoothed with a bilateral filter, which preserves edges inside the ROI. A global threshold is then selected using Otsu's method, producing a hard binary mask from the soft score map. The script finally saves both the soft mask (`mask_soft.png`) and the hard mask (`mask_hard_otsu.png`), together with a visualization of the rectangular ROI used during the computation. The soft mask (`mask_soft.png`) is then used at runtime as a per-pixel weighting function inside the NCC computation:

$$s_{\text{pos}} = \text{NCC}(I, \mu_{\text{pos}}; \text{mask}), \qquad s_{\text{neg}} = \text{NCC}(I, \mu_{\text{neg}}; \text{mask}),$$

so that only the masked region contributes to the final LLR score. In practice, the most effective mask was obtained by running the script on a curated subset of images (`pos_mask` and `neg_mask`) in which the yarn color provided contrast against the background. Among the three yarn colors available in the dataset (light blue, purple, and fuchsia). The fuchsia yarn was specifically included to demonstrate that the proposed method does not rely on RGB color contrast. Although its color is visually close to the pink machine background, the preprocessing pipeline (grayscale conversion, CLAHE, and ROI masking) makes the detector largely invariant to chromatic differences, allowing fuchsia samples to be handled as reliably as more visually distinct yarns.

(a) `mask_soft.png` **(b)** `mask_hard_otsu.png`

Fig. 8. Visualization of the ROI used during the computation

E. Log-Likelihood Ratio Classification. For each cropped image, the classifier computes two normalized cross-correlation (NCC) scores:

$$s_{\text{pos}}, \quad s_{\text{neg}},$$

corresponding to the similarity between the current frame and the positive and negative prototypes, respectively. A binary ROI mask is applied to restrict the computation to the relevant hook area.

The decision variable is defined as:

$$LLR = s_{\text{pos}} - s_{\text{neg}}. \qquad [1]$$

A threshold of LLR was selected to minimize false negatives, which are more detrimental in this application than false positives. If the threshold is not satisfied, the system classifies the hook as defective and triggers an immediate stop command to the machine.

F. Code Availability. The complete implementation—including synchronized image capture, ROI extraction, preprocessing, prototype generation, and real-time LLR evaluation—is available in the project repository: https://github.com/AlessiaRinaldi/automated_knitting_machine.

5. Experimental Evaluation and Results

A. Experimental Protocol. The performance of the proposed LLR-based detector was evaluated within the binary hypothesis-testing framework commonly adopted in supervised signal detection:

$$H_0 : \text{"U-shape" of the yarn}, \qquad H_1 : \text{hole or missing yarn}.$$

At each microswitch activation, the system acquires a cropped hook image, computes the log-likelihood ratio:

$$LLR = s_{\text{pos}} - s_{\text{neg}},$$

and compares it to a fixed threshold to decide between H_0 and H_1. This setting is equivalent to a classical detector with probabilities of detection and false alarm P_D and P_F, as described in the course notes on supervised signal detection and ROC analysis.

To obtain an unbiased estimate of performance, the available dataset was randomly partitioned into three non-overlapping subsets:

- **Training set**, used exclusively to build the positive and negative prototypes and the discriminative ROI mask. It consists of 39 positive images and 27 negative images, balanced across the three yarn colors (13 positive and 9 negative crops per color).
- **Validation set**, containing 208 images, used only for threshold selection (LLR) and operating-point tuning.
- **Test set**, containing 771 images, used only for the final performance evaluation of the fixed detector.

B. Threshold Selection with Validation Set. During the validation phase, the classifier was evaluated on the entire validation set using $LLR = 0$. With this configuration, at least one false alarm (false positive) occurred. Such an event is considered unacceptable in the real operating scenario: actual defects are extremely rare, and even a single false alarm would trigger an unnecessary machine stop, resulting in a higher operational cost than missing an occasional defect.

Following a Neyman–Pearson style design criterion, the decision threshold was therefore adjusted to obtain a false-alarm rate *as close as possible to zero* on the validation set, even at the expense of an increased number of false negatives, which are less critical for this application. This procedure led to the selection of the threshold $\text{LLR}_{\text{thr}} = -0.035$, which was subsequently kept fixed for the final evaluation on the independent test set.

C. Test-Set Performance. Once the threshold was fixed, the detector was evaluated on the independent test set of 771 images. Table 1 reports the confusion matrix aggregated over all yarn colors.

Table 1. Confusion matrix on the test set (771 images)

	Predicted H_0	**Predicted** H_1
True H_0 (no defect)	TN = 633	FP = 0
True H_1 (defect)	FN = 20	TP = 118

From these values, the main performance indicators are:

$$\text{Accuracy} = \frac{TP+TN}{TP+TN+FP+FN} = \frac{751}{771} \approx 97.4\%,$$
$$P_D \text{ (TPR)} = \frac{TP}{TP+FN} = \frac{118}{138} \approx 85.5\%,$$
$$P_F \text{ (FPR)} = \frac{FP}{FP+TN} \approx 0\%,$$
$$\text{Specificity} = \frac{TN}{TN+FP} \approx 100\%.$$

Thus, the chosen operating point corresponds to a detector that never raises false alarms on the test set and correctly identifies about 85% of defective hooks. In terms of detection-theory terminology, the design enforces $P_F \approx 0$ and accepts a moderate increase in missed detections $P_M = 1 - P_D$ to satisfy this constraint. This is consistent with a minimum-risk perspective in which the cost of stopping the machine unnecessarily is much higher than the cost of letting a small number of defective hooks pass undetected.

D. Per-Color Performance Analysis. A more detailed analysis was conducted by grouping the test images according to the yarn color (light blue, purple, fuchsia).

Table 2. Fuschia Confusion matrix on the test set (373 images)

	Predicted H_0	Predicted H_1
True H_0	TN = 322	FP = 0
True H_1	FN = 3	TP = 48

Fuschia performance indicators:

$$\text{ACC} \approx 99.2\%, \qquad \text{TPR} \approx 94.12\%, \qquad \text{FPR} \approx 0\%.$$

Table 3. Purple Confusion matrix on the test set (238 images)

	Predicted H_0	Predicted H_1
True H_0	TN = 193	FP = 0
True H_1	FN = 8	TP = 37

Purple performance indicators:

$$\text{ACC} \approx 96.63\%, \qquad \text{TPR} \approx 82.2\%, \qquad \text{FPR} \approx 0\%.$$

Table 4. Light blue Confusion matrix on the test set (160 images)

	Predicted H_0	Predicted H_1
True H_0	TN = 118	FP = 0
True H_1	FN = 9	TP = 33

Light Blue performance indicators:

$$\text{ACC} \approx 94.37\%, \qquad \text{TPR} \approx 78.57\%, \qquad \text{FPR} \approx 0\%.$$

As anticipated, the best performance is obtained for the **fuchsia** yarn, even though this color is visually the most similar to the pink background of the machine. Conversely, the **light blue** yarn yields the lowest detection probability, because in grayscale it is the closest to the background and therefore generates less contrast in the ROI used by the NCC-based LLR. The purple yarn lies between these two extremes.

This behavior can be explained by the way the prototypes and the discriminative mask are estimated: the separability between classes does not depend on raw chromatic contrast but on how stable the local texture and intensity patterns are after preprocessing and grayscale conversion. For the fuchsia yarn, the combination of CLAHE, ROI restriction, and prototype-based NCC generates more consistent differences between H_0 and H_1 than for the light blue yarn, despite the lower apparent color contrast in the original RGB images.

Overall, the experiments indicate that the proposed mask-based LLR detector achieves high detection accuracy while maintaining a very low false alarm rate at the selected operating point, supporting its suitability for inline deployment on the crochet machine. Future work will investigate more advanced feature representations (e.g., learned descriptors or CNN-based embeddings) to further improve P_D, especially in the most challenging low-contrast conditions.

E. Out-of-distribution failure case (pale pink yarn). The pale pink yarn was not included in the training, validation, or test sets, as it does not belong to the normal operating conditions of the machine. In this additional edge case, more than 32% of the samples were misclassified, indicating that the detector is unable to operate reliably when yarn and background exhibit nearly identical grayscale intensities. In this regime, the LLR distributions of H_0 and H_1 overlap almost completely, confirming that such an extreme low-contrast condition lies outside the operational domain of the method.

These observations reinforce the need for future extensions based on learned feature representations or contrast-invariant descriptors.

The full edge case dataset and numerical results are provided in the project repository.

F. Real-Time Operation on Raspberry Pi 3. The complete acquisition and detection pipeline runs directly on the Raspberry Pi 3 and is triggered by the microswitch signal. In the current mechanical configuration, the microswitch is activated approximately once every 0.42 s, so that each hook is inspected at a rate of about

$$f_{\text{inspect}} = \frac{1}{0.42} \approx 2.4 \text{ inspections per second.}$$

At each activation, the software captures a full-resolution still image (1280×960 pixels), immediately crops a small region of interest around the hook (about 76×76 pixels), and runs the preprocessing, NCC-based similarity, and LLR computation on the crop. From a computational point of view, the cost of the LLR stage is negligible compared to the image capture time: preprocessing (grayscale conversion, Gaussian filtering, CLAHE) and two NCC evaluations on a 76×76 patch require only a few milliseconds on a Raspberry Pi 3. As a result, the overall processing time per trigger comfortably fits within the 0.42 s interval imposed by the machine cycle, and the effective inspection rate is limited by the mechanics rather than by the classifier.

The large computational margin available on the Raspberry Pi 3 also enables the exploration of multi-sample strategies aimed at improving the detection probability. Since the mechanical cycle provides approximately 0.42 s between two inspections, several images of the same hook could be acquired either within a short temporal window or over consecutive cycles without violating the real-time constraint.

Such redundancy can be exploited through simple decision-fusion schemes, such as majority voting or averaging the LLR scores across multiple samples. Importantly, these strategies are expected to reduce the number of missed detections (i.e., lower P_M) without increasing the false-alarm probability, which must remain $P_F \approx 0$ for safe operation. In a Neyman–Pearson perspective, additional samples effectively sharpen the separation between the H_0 and H_1 score distributions, allowing more reliable decisions while strictly respecting the zero–false-alarm requirement of the application.

Investigating these multi-sample fusion techniques is therefore a promising direction for improving robustness, particularly in the most challenging low-contrast configurations where single-frame decisions may occasionally produce false negatives.

No triggers were missed during testing, and the stop signal was reliably issued on the first defective cycle. These results confirm that the proposed system meets real-time requirements and is suitable for continuous inline monitoring of automated crochet machines.

6. Conclusions and Future Work

This work presented a complete real-time vision system for hook-level error detection on an automated crochet machine, implemented on a Raspberry Pi 3 using a lightweight LLR-based classifier. The proposed pipeline combines event-driven image acquisition, ROI extraction, preprocessing, and prototype-based NCC evaluation, achieving fast and deterministic operation compatible with the mechanical constraints of the machine. The system attains high accuracy with an operating point tending towards zero false alarms, which is essential for industrial applicability where unnecessary machine stoppages must be avoided. On the test set of 771 images, the detector reached an overall accuracy of approximately 97.4% with a detection probability of about 85%.

Several directions for future work emerge from these findings. First, the significant computational margin on the Raspberry Pi 3 suggests that multi-sample decision fusion (e.g., majority voting or averaging LLR scores across several frames) could reduce missed detections while maintaining the critical constraint $P_F \approx 0$. Second, while the current method relies on hand-crafted prototypes and a static ROI mask, more advanced feature representations, such as convolutional autoencoders, learned embeddings, or lightweight CNN classifiers, may provide improved robustness under challenging illumination and low-contrast scenarios. Third, improving mechanical stability and camera alignment would further increase repeatability of the ROI crop, enabling a more stable decision boundary over long-term operation.

Overall, the results demonstrate that low-cost embedded vision can support reliable, real-time quality control on crochet machines. Extending the approach with multi-frame fusion and learned feature extractors represents a promising avenue toward even higher detection reliability.